

# Mock Design: A Web Crawler

Phase 4 · Interview-Ready · Estimated time: 1.5 hours

---

## 1. Concept From First Principles: Applying the Framework

The web crawler is the final classic mock-design problem in this Phase, and deliberately closes out the "named canonical problems" set (Chapters 3–17) because it combines nearly every category of concern covered so far — queueing at scale, deduplication, rate limiting, and distributed coordination — into a single system whose entire purpose is exploring an effectively infinite, constantly-changing graph (the web) in a bounded, well-behaved way.

### Step 1 — Clarify requirements

- Functional: given a set of seed URLs, discover and fetch web pages, extract new links from each page, and continue crawling discovered URLs, storing fetched content for downstream use (e.g., search indexing).
- Non-functional: must not overwhelm any single website (politeness); must avoid re-crawling the exact same URL repeatedly without reason; must scale to billions of pages; must tolerate individual page/site failures gracefully without stopping the overall crawl.

### Step 2 — Estimate scale

Assume a target of crawling 1 billion pages per month, giving roughly 400 pages/second average sustained fetch rate. The real design challenge isn't this raw throughput number alone — it's that those 400 fetches/second must be spread across millions of distinct websites **without exceeding any single site's** tolerable request rate, which is a fundamentally different constraint than any earlier chapter's uniform rate limiting.

### Step 3 — API / interface

This system has no end-user-facing API in the usual sense — it's an internal batch/continuous pipeline. The relevant "interface" is the URL Frontier's enqueue/dequeue contract: `enqueue(url, priority)`, `dequeue()` -> url (respecting politeness, detailed in Step 5), plus a `downstream_store(url, content, extractedLinks)` call after each fetch.

### Step 4 — High-level design

Seed URLs → **URL Frontier** (the queue of URLs waiting to be crawled) → a pool of **Fetcher** workers pull URLs, download the page, and pass it to a **Parser** which extracts the page's content (for storage) and any new links (fed back into the Frontier). A **URL Seen/Dedup Store** checks every newly-extracted link against previously-crawled or already-queued URLs before enqueueing, preventing the same URL from being processed repeatedly. This is a continuous, cyclical pipeline, not a linear one — new links constantly feed back into the same Frontier that's being drained.

### Step 5 — Deep dive: politeness and URL deduplication at scale

**Politeness** is the single most commonly probed, and most commonly under-addressed by weaker candidates, part of this problem — hammering one website with thousands of simultaneous requests is both a bad internet citizen and likely to get your crawler's IP blocked entirely. The standard solution: structure the URL Frontier as **per-host queues** rather than one global queue — a front-queue layer prioritizes which hosts to crawl next, while a back-queue layer maintains one dedicated queue per host with an enforced minimum delay between requests to that specific host (directly reusing Chapter 4's rate-limiting concepts, just applied per-destination instead of per-client). This also means respecting each site's robots.txt file, which explicitly states which paths a well-behaved crawler should avoid — a real, standard web convention worth naming unprompted.

**URL deduplication at billions-of-URLs scale** is the second major deep-dive direction. Storing every seen URL in a traditional database or exact hash set becomes prohibitively memory-expensive at this cardinality. The standard, well-justified answer is a **Bloom filter** (Phase 3, Ch 10): a space-efficient probabilistic structure that can definitively say "this URL has never been seen" or "this URL has probably been seen" (with a small, tunable false-positive rate), which is an acceptable trade here — occasionally skipping a URL that was actually new (a false positive) is a minor, acceptable loss, whereas the alternative (exact tracking) would require far more memory than is practical at billions of URLs.

## Step 6 — Trade-offs and wrap-up

Volunteer: a naive crawler can fall into **crawler traps** — pages that generate effectively infinite new URLs (e.g., a calendar page with an infinite sequence of "next month" links) — wasting crawl budget forever on a single low-value site. Name a mitigation: cap the maximum crawl depth or total URL count per host, and monitor (Phase 3, Ch 13) for hosts consuming disproportionate crawl budget relative to the value of content discovered.

## 2. Real-World Example / Case Study

Googlebot's publicly documented crawling behavior — respecting robots.txt, enforcing per-host crawl rate limits, and prioritizing pages by estimated importance — is the most widely known real-world instance of exactly this design, and per-host politeness in particular is a well-documented, non-negotiable requirement for any production-scale crawler, not an interview simplification.

## 3. Diagram

```

[Seed URLs] -> [URL Frontier]
      |
      front queues (priority) -> back queues (per-host)
host A: [url1, url2, ...]    min delay between fetches
host B: [url3, ...]
host C: [url4, url5, ...]
      |
      [Fetcher Workers] (pool, respects per-host pacing)
      |
      [Parser] -> extract content -> [Content Store]
                -> extract links
                |
                check against [Bloom Filter: seen URLs?]
                new -> enqueue back into Frontier
                seen (probably) -> discard

robots.txt checked before enqueueing any URL from a new host.

```

## 4. Trade-offs Table

| Decision                            | Alternative                          | Why this choice                                                                       |
|-------------------------------------|--------------------------------------|---------------------------------------------------------------------------------------|
| Per-host queues with enforced delay | One global queue, no per-host pacing | Prevents overwhelming any single site; avoids IP bans, respects politeness            |
| Bloom filter for URL dedup          | Exact hash set / database lookup     | Space-efficient at billions of URLs; small false-positive rate is an acceptable trade |
| Cap crawl depth/URL count per host  | Unlimited crawling per host          | Prevents crawler traps from consuming disproportionate crawl budget                   |
| Continuous, cyclical pipeline       | One-time linear batch crawl          | The web constantly changes; new links must keep feeding back into the Frontier        |

## 5. Common Interview Questions

### Q1: "How do you avoid overwhelming any single website with crawl requests?"

Model approach: Describe per-host back-queues with an enforced minimum delay directly, and name robots.txt compliance as an additional, standard web-convention requirement worth respecting.

### Q2: "How do you track billions of already-crawled URLs without running out of memory?"

Model approach: Name the Bloom filter directly, and explain the specific trade being accepted — a small false-positive rate (occasionally skipping a genuinely new URL) in exchange for dramatically lower memory usage than exact tracking.

### Q3: "What's a crawler trap, and how do you defend against one?"

Model approach: Describe a page generating effectively infinite new URLs (like an infinite calendar), and name capping crawl depth or per-host URL budget as the direct mitigation.

## 6. Practice Exercises

- 1 Run the full six-step framework for "design a web crawler," out loud, with a timer.
- 2 Design the front-queue/back-queue URL Frontier structure in more detail: how does a host get selected from the front queue, and how is per-host pacing enforced in the back queue?
- 3 Estimate the memory footprint of a Bloom filter sized for 10 billion URLs at a 1% false-positive rate, and compare it conceptually to an exact hash set's memory cost at the same scale.

## 7. Curated YouTube Videos

| Language | Video & Channel                                                                           | Why it helps                                                                                       |
|----------|-------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------|
| English  | "System Design Interview - Design a Web Crawler (Full mock interview with Sr. MAANG SWE)" | Full live mock interview matching this chapter's exact structure.                                  |
| English  | "Design a Web Crawler System Design Interview w/ a Ex-Meta Staff Engineer"                | Covers politeness and URL frontier design in depth.                                                |
| Hindi    | "System Design Complete Course 2026" (Hindi + English, 10+ hrs)                           | Bilingual course covering crawling, queueing, and Bloom filters as part of the broader curriculum. |
| Hindi    | "Complete System Design Roadmap 2026   HLD & LLD" — CodeHelp                              | Hindi roadmap reinforcing probabilistic data structure fundamentals this chapter builds on.        |

## 8. Key Takeaways

- The web crawler combines queueing, deduplication, rate limiting, and distributed coordination from across this entire program into one system.
- Politeness (per-host pacing and robots.txt compliance) is the single most commonly probed, and most often under-addressed, part of this problem.
- Per-host queues with enforced delays apply Chapter 4's rate-limiting concepts per-destination rather than per-client.
- Bloom filters make URL deduplication tractable at billions-of-URLs scale, trading a small false-positive rate for massive memory savings.
- Crawler traps (infinite-URL-generating pages) require explicit depth/budget caps to avoid wasting crawl resources.
- This is a continuous, cyclical pipeline, not a linear batch job — newly discovered links constantly feed back into the same queue being drained.
- This chapter closes out the set of classic named mock-design problems in this Phase, having now touched nearly every architectural pattern taught across all four Phases.